SenEvent

apps/web · **apps/mobile** · packages/shared

DAWM · BLOC MOBILE 2/3 · M1 RETEL — DMI/FST/UCAD

Lab 9 — Expo Router et navigation mobile

Rapport de réalisation : l'application mobile passe d'un seul écran à une vraie navigation. Une barre d'onglets (Événements, Profil), un écran de détail empilé qui lit l'id de l'événement dans l'URL, et des cartes qui y mènent d'un toucher. Ce sont les concepts de React Router du Lab 4, traduits dans le vocabulaire d'Expo Router — et toujours `getEvenements()` du package partagé, sans requête réécrite.

tag : v0.9-router | Expo Router 6 · SDK 54 | Stack + Tabs | evenement/[id].js | useLocalSearchParams | 0 appel Supabase en dur

01 Ce que le Lab 8 avait laissé

À la fin du Lab 8, l'application affichait la liste des événements, mais tout tenait dans un seul fichier `App.js` : impossible d'ouvrir un événement, impossible d'atteindre un écran de profil. Une application mobile a plusieurs écrans et une barre d'onglets pour circuler entre ses sections principales.

Ce lab n'ajoute pas de logique métier. Il réorganise l'application en écrans, et le seul appel de données reste `getEvenements()` — réutilisé, cette fois, par deux écrans.

02 Le système de fichiers est la configuration

Avec Expo Router, on ne déclare plus de routes dans du code : chaque fichier du dossier `app/` devient un écran, et un fichier `_layout.js` décide comment ses voisins s'articulent. Deux conventions de nommage suffisent : les parenthèses de `(tabs)` créent un **groupe** qui n'ajoute rien à l'URL (l'onglet liste reste à `/`), et les crochets de `[id]` déclarent un **paramètre**.

BESOIN	WEB — LAB 4 (REACT ROUTER)	MOBILE — LAB 9 (EXPO ROUTER)
Declarer une route	`<Route path="/" element={…} />`	`un fichier dans app/`
Route parametree	`/evenement/:id`	`app/evenement/[id].js`
Lire le parametre	`useParams()`	`useLocalSearchParams()`
Naviguer par code	`useNavigate()`	`useRouter()`
Lien cliquable	`<Link to={…}>`	`<Link href={…}>`
Point de montage	`<BrowserRouter> dans App.jsx`	`app/_layout.js`
Logique metier	**`getEvenements() de @senevent/shared`**	**`getEvenements() de @senevent/shared`**

La dernière ligne est la même des deux côtés : c'est l'architecture du cours qui tient. La seule idée réellement nouvelle est que la configuration passe par l'arborescence plutôt que par du JSX.

03 Deux navigateurs imbriqués : Stack et Tabs

Le layout racine est un `Stack` : les écrans s'empilent, avec un retour automatique. Il contient le groupe des onglets et l'écran de détail — toucher une carte pousse le détail *au-dessus* des onglets. C'est aussi le premier fichier exécuté : il prend le rôle d'initialisation de Supabase qu'avait `App.js`.

APPS/MOBILE/APP/_LAYOUT.JS

```
initSupabase(
  process.env.EXPO_PUBLIC_SUPABASE_URL,
  process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY
);

// Ouvert directement par un lien profond (senevent://evenement/3), le detail
// garde les onglets en dessous de lui : la fleche retour ramene a la liste
// au lieu de fermer l'app.
export const unstable_settings = {
  initialRouteName: "(tabs)",
};

export default function RootLayout() {
  return (
    <SafeAreaProvider>
      <StatusBar style="light" />
      <Stack
        screenOptions={{
          headerStyle: { backgroundColor: couleurs.fondEntete },
          headerTintColor: couleurs.texte,
          headerTitleStyle: { fontWeight: "700" },
          headerShadowVisible: false,
          headerBackTitle: "Retour",
          // Fond sombre pendant les transitions : sans lui, React Navigation
          // affiche son fond clair par defaut le temps de l'animation.
          contentStyle: { backgroundColor: couleurs.fond },
        }}
      >
        <Stack.Screen name="(tabs)" options={{ headerShown: false }} />
        <Stack.Screen name="evenement/[id]" options={{ title: "Detail" }} />
      </Stack>
```

Le groupe `(tabs)` a son propre layout, un navigateur `Tabs`. Les icônes viennent de `@expo/vector-icons` ; la fonction `tabBarIcon` reçoit la couleur et l'état de l'onglet, ce qui permet d'afficher la version pleine de l'icône quand l'onglet est actif.

APPS/MOBILE/APP/(TABS)/_LAYOUT.JS — UN ONGLET

```
      <Tabs.Screen
        name="index"
        options={{
          title: "Evenements",
          // L'onglet garde son en-tete de marque du Lab 8 (degrade + compteur),
          // qui joue deja le role de barre de titre.
          headerShown: false,
          tabBarIcon: ({ color, size, focused }) => (
            <Ionicons
              name={focused ? "calendar" : "calendar-outline"}
              size={size}
              color={color}
            />
```

L'onglet Événements masque l'en-tête du navigateur pour garder l'en-tête de marque du Lab 8 (dégradé et compteur). C'est donc à lui de laisser la place à l'encoche : il lit la hauteur avec `useSafeAreaInsets`, qui fonctionne sur iOS comme sur Android, là où le Lab 8 compensait la barre d'état Android à la main.

04 La carte devient un lien

`<Link href>` est le pendant du `<Link to>` du web, et le chemin se construit de la même façon. Avec `asChild`, `Link` ne dessine rien : il transmet son comportement de navigation à son enfant, un `Pressable`, et toute la carte devient touchable.

```
    <Link href={`/evenement/${evenement.id}`} asChild>
      <Pressable>
        {({ pressed }) => (
          <View style={[styles.carte, pressed && styles.carteEnfoncee]}>
```

Un piège rencontré ici : `Link` fusionne les styles de son enfant comme des objets. Un style écrit sous forme de fonction, `style={({ pressed }) => …}`, serait silencieusement perdu. Le retour visuel du toucher passe donc par une fonction *enfant* du `Pressable`, qui reçoit le même état `pressed`.

05 L'écran de détail lit son paramètre

Le nom `[id].js` déclare le paramètre ; `useLocalSearchParams()` le lit — l'exact pendant de `useParams()`. La valeur est toujours une chaîne : `/evenement/3` donne `"3"`, d'où le `Number(id)` avant de comparer.

```
  const { id } = useLocalSearchParams();
  const [evenement, setEvenement] = useState(null);
  const [chargement, setChargement] = useState(true);
  const [erreur, setErreur] = useState(null);
  const [imageCassee, setImageCassee] = useState(false);

  useEffect(() => {
    // L'ecran ne recoit pas la liste de l'onglet Evenements : il rappelle la
    // fonction partagee et cherche l'id. Aucune requete Supabase ecrite ici.
    const charger = async () => {
      try {
        const tous = await getEvenements();
        setEvenement(tous.find((ev) => ev.id === Number(id)) ?? null);
      } catch (e) {
        setErreur(e.message);
      } finally {
        setChargement(false);
      }
    };
    charger();
  }, [id]);
```

Au Lab 4, la page Détail recevait le tableau en prop. Ici chaque écran est indépendant et ne reçoit pas l'état de l'onglet liste : il rappelle donc la fonction partagée et cherche l'événement. Le parcours mesuré le confirme : l'ouverture du détail ajoute exactement 1 requête. Trois états sont traités : chargement, erreur réseau, et identifiant inconnu (figure 05).

06 Bonus : naviguer par code

`useRouter` est l'équivalent de `useNavigate`. Le sujet propose `router.push("/")` pour le bouton « Retour à la liste ». Dans Expo Router 6, `push` émet toujours une action `PUSH` : on empilerait une seconde copie des onglets par-dessus le détail, et la flèche retour ramènerait… au détail. On utilise `dismissTo("/")`, qui dépile jusqu'à la liste — ou la met à la place du détail si on est arrivé par un lien direct.

```
function BoutonRetourListe() {
  const router = useRouter();
  return (
    <Pressable
      onPress={() => router.dismissTo("/")}
      style={({ pressed }) => [styles.bouton, pressed && styles.boutonEnfonce]}
    >
      <Ionicons name="list" size={16} color={couleurs.fond} />
      <Text style={styles.boutonTexte}>Retour a la liste</Text>
    </Pressable>
```

Le parcours le vérifie : après le bouton, l'application est sur `/` et il reste 0 flèche retour visible — la pile est propre.

07 Metro et les dépendances d'expo-router

Le sujet indique que le `metro.config.js` du Lab 8 reste valable. Dans ce dépôt, ce n'était pas le cas : le premier bundle a échoué sur `Unable to resolve module @expo/metro-runtime`. npm a rangé plusieurs dépendances d'expo-router dans des `node_modules` imbriqués (`expo-router/node_modules/@react-navigation/bottom-tabs/node_modules/…`), que le réglage `disableHierarchicalLookup` du Lab 8 empêchait de voir.

Réactiver simplement la recherche hiérarchique aurait été pire : la racine du monorepo contient d'autres versions de `react` et `react-native`, et les paquets hissés comme `react-native-screens` les auraient trouvées — deux React dans le même bundle. La règle retenue :

```
// 3. Les versions de l'app passent avant celles de la racine.
//    npm a hisse a la racine d'autres versions de react, react-native et expo
//    (tirees par les peer dependencies). Un paquet hisse comme
//    react-native-screens trouverait ces copies-la en remontant l'arborescence :
//    deux React dans le meme bundle, et l'app plante.
//    Au Lab 8, on coupait donc toute remontee (disableHierarchicalLookup). Mais
//    expo-router range ses propres dependances dans des node_modules imbriques
//    (expo-router/node_modules/@react-navigation/bottom-tabs/node_modules/...)
//    que Metro ne voyait alors plus. D'ou la regle : resolution normale, sauf
//    pour un paquet installe dans apps/mobile, qui est toujours pris la.
const installeDansApp = new Map();
const nomDuPaquet = (module) =>
  module.split("/").slice(0, module.startsWith("@") ? 2 : 1).join("/");

config.resolver.resolveRequest = (context, moduleName, platform) => {
  const estUnPaquet = !moduleName.startsWith(".") && !path.isAbsolute(moduleName);
  if (estUnPaquet) {
    const paquet = nomDuPaquet(moduleName);
    if (!installeDansApp.has(paquet)) {
      installeDansApp.set(paquet, fs.existsSync(path.join(modulesApp, paquet)));
    }
    if (installeDansApp.get(paquet)) {
      // On resout comme si l'import venait d'un fichier de l'app elle-meme.
      return context.resolveRequest(
        { ...context, originModulePath: path.join(projectRoot, "package.json") },
        moduleName,
        platform
      );
    }
  }
  return context.resolveRequest(context, moduleName, platform);
};
```

Le contrôle est fait sur la sourcemap du bundle Android : `react` vient uniquement de `/apps/mobile/node_modules` et `react-native` de `/apps/mobile/node_modules`.

08 Structure livrée

```
apps/mobile/
  app/
    _layout.js              # Stack racine + initSupabase
    (tabs)/
      _layout.js            # navigateur Tabs (2 onglets + icones)
      index.js              # onglet Evenements : FlatList, filtres, etats
      profil.js             # onglet Profil (placeholder Lab 10)
    evenement/
      [id].js               # detail : useLocalSearchParams + bonus useRouter
  components/
    EvenementCarte.js       # carte cliquable (Link asChild)
    EnteteAccueil.js        # en-tete de marque, zone sure via useSafeAreaInsets
    FiltreCategories.js · CarteSquelette.js
  theme.js · format.js
  metro.config.js           # resolution monorepo (revue pour Expo Router)
  app.json                  # scheme "senevent" + plugin expo-router
  package.json              # main : expo-router/entry
```

09 Ce que la validation a réellement donné

Les vérifications de l'enseignant (section 9.2) et les bundles natifs, relancés au moment de générer ce rapport :

CONTRÔLE	COMMANDE / MÉTHODE	RÉSULTAT	OK
L'ancien App.js est supprime	`test ! -f apps/mobile/App.js`	`supprime`	✓
Point d'entree Expo Router	`"main" de apps/mobile/package.json`	`expo-router/entry`	✓
Schema d'application declare	`"scheme" de app.json`	`senevent`	✓
Le groupe (tabs) contient les onglets	`ls "apps/mobile/app/(tabs)/"`	`_layout.js index.js profil.js`	✓
La route parametree existe	`test -f "apps/mobile/app/evenement/[id].js"`	`OK`	✓
Le detail lit l'id de l'URL	`grep -rl useLocalSearchParams apps/mobile/app`	`apps/mobile/app/evenement/[id].js`	✓
Aucune logique Supabase reecrite	`git grep "supabase.from|createClient" -- apps/mobile`	`0 occurrence`	✓
Le .env mobile n'est pas versionne	`git ls-files | grep -c "apps/mobile/.env$"`	`0 fichier suivi`	✓
Bundle Android	`npx expo export --platform android`	`1148 modules`	✓
Bundle iOS	`npx expo export --platform ios`	`1152 modules`	✓
Une seule copie de React dans le bundle	`sources de la sourcemap Android`	`react : /apps/mobile/node_modules · react-native : /apps/mobile/node_modules`	✓

Le parcours de navigation, piloté par `apps/mobile/capture-lab9.mjs` sur l'application réelle et la vraie base :

CONTRÔLE	MÉTHODE	RÉSULTAT	OK
L'app demarre sur l'onglet Evenements	`chemin au lancement`	`/`	✓
Deux onglets dans la barre du bas	`liens role=tab`	`Evenements → / · Profil → /profil`	✓

CONTRÔLE	MÉTHODE	RÉSULTAT	OK
Toucher une carte ouvre son detail	`toucher /evenement/1`	`/evenement/1`	✓
Le detail affiche l'evenement de l'URL	`titre et lieu compares a la reponse Supabase`	`Concert acoustique de Youssou Junior`	✓
Le detail rappelle getEvenements()	`requetes /rest/v1/evenements a l'ouverture`	`+1`	✓
La fleche retour ramene a la liste	`toucher la fleche du Stack`	`/`	✓
L'onglet Profil s'affiche	`toucher l'onglet Profil`	`/profil`	✓
Chaque onglet garde son etat	`requetes en revenant sur Evenements`	`0 rechargement`	✓
Lien direct vers un detail	`ouvrir /evenement/8 puis retour`	`/`	✓
Identifiant inconnu	`ouvrir /evenement/999999`	`Evenement introuvable.`	✓
Bonus : Retour a la liste par code	`depuis /evenement/2`	`/ · 0 fleche restante`	✓
Aucune erreur JavaScript	`pageerror pendant tout le parcours`	`0 erreur`	✓
La donnee vient bien de Supabase	`domaines contactes`	`supabase.co contacte`	✓

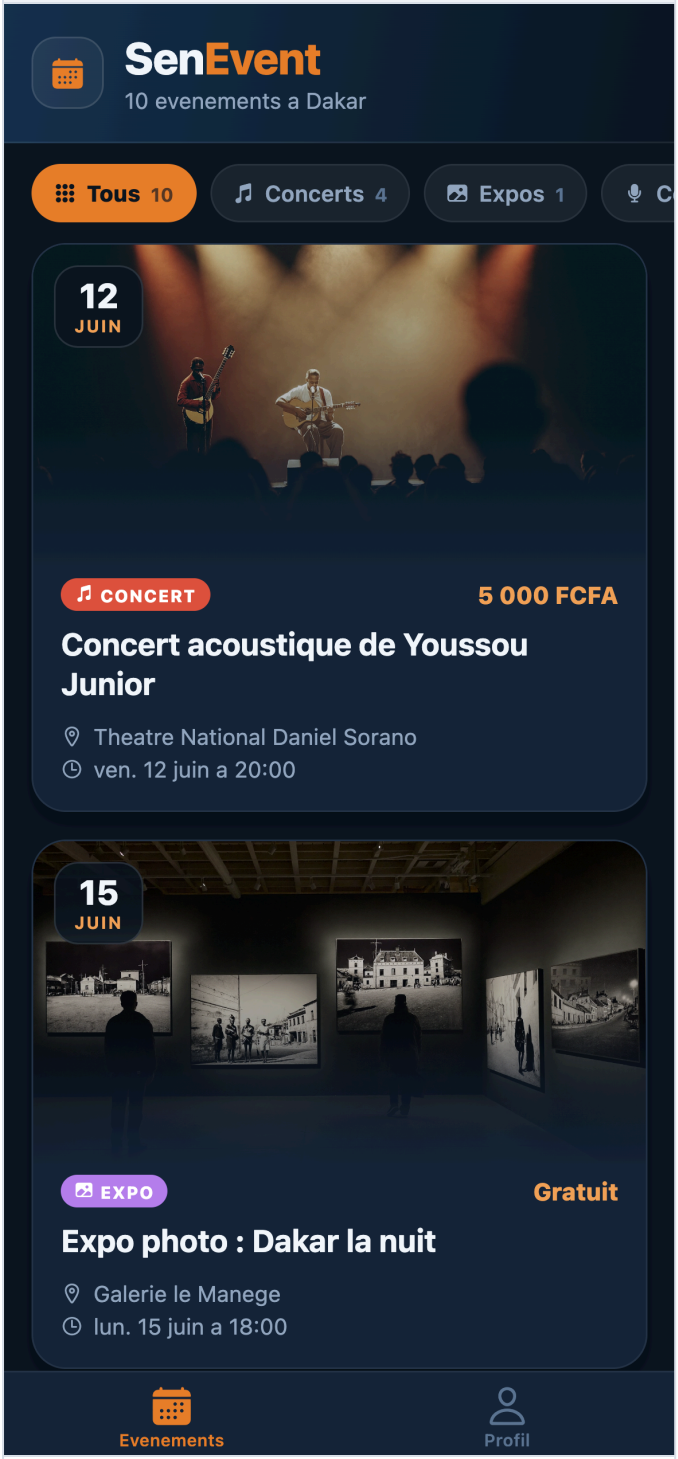
Le lab compte 6 commits depuis `v0.8-expo`, dans l'ordre de merge imposé : installation du routeur et configuration (A), onglets (B), détail et cartes cliquables (C), puis le bonus.

10 L'application, en situation

Captures au format téléphone, avec la vraie base. C'est le même dossier `app/` que celui exécuté dans Expo Go, rendu sur la cible web d'Expo pour pouvoir piloter le parcours par script — avec un bénéfice propre à Expo Router : chaque écran a une URL, visible en tête de chaque figure.

FIGURE 01 `/`

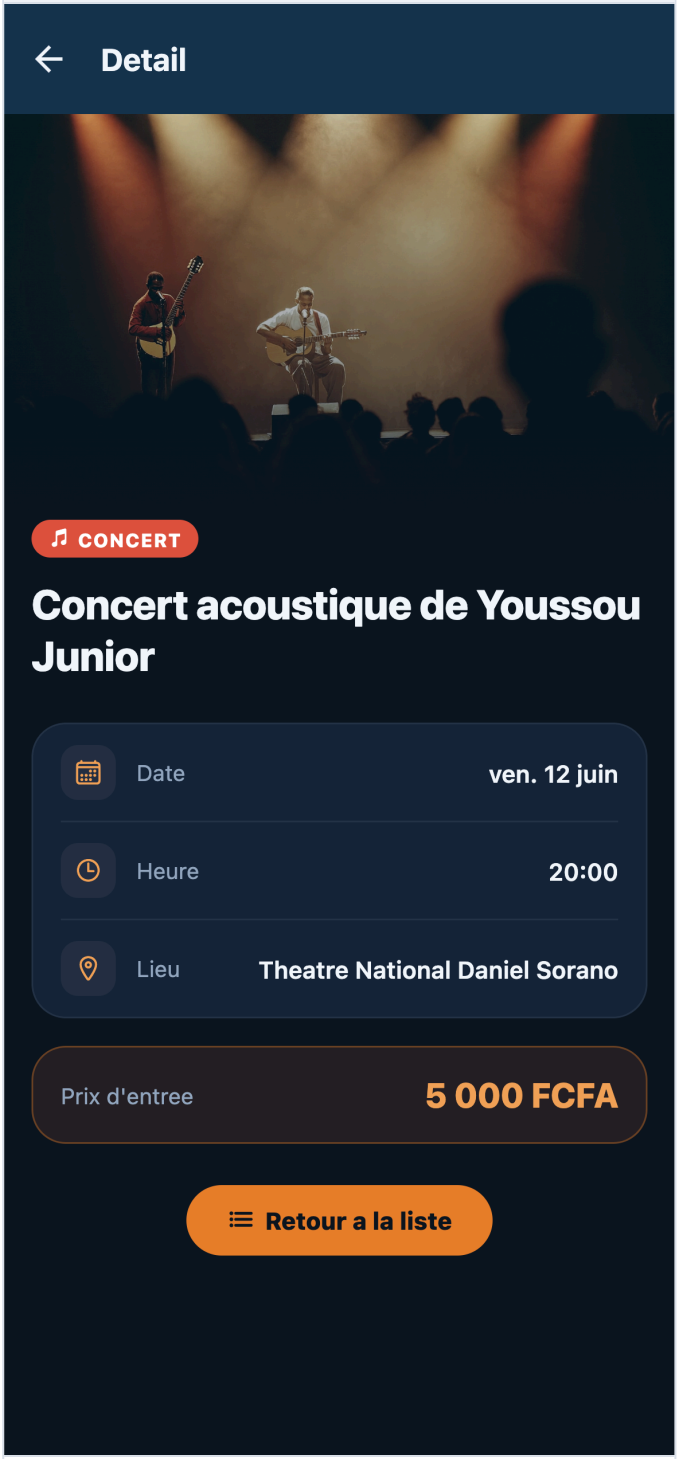
Onglet Evenements



La liste du Lab 8 vit maintenant dans app/(tabs)/index.js. La barre d'onglets en bas vient du layout Tabs ; les 10 cartes sont des liens vers /evenement/:id.

FIGURE 02 `/evenement/1`

Detail apres un toucher



Le detail est empile au-dessus des onglets par le Stack racine, avec sa fleche retour. L'id 1 est lu dans l'URL par useLocalSearchParams, l'evenement retrouve dans getEvenements(). En bas, le bouton du bonus.

FIGURE 03 `/profil`

Onglet Profil



Placeholder : la connexion arrive au Lab 10. L'en-tete est celui du navigateur Tabs ; l'icone de l'onglet actif passe en orange et en version pleine.

FIGURE 04 `/evenement/8`

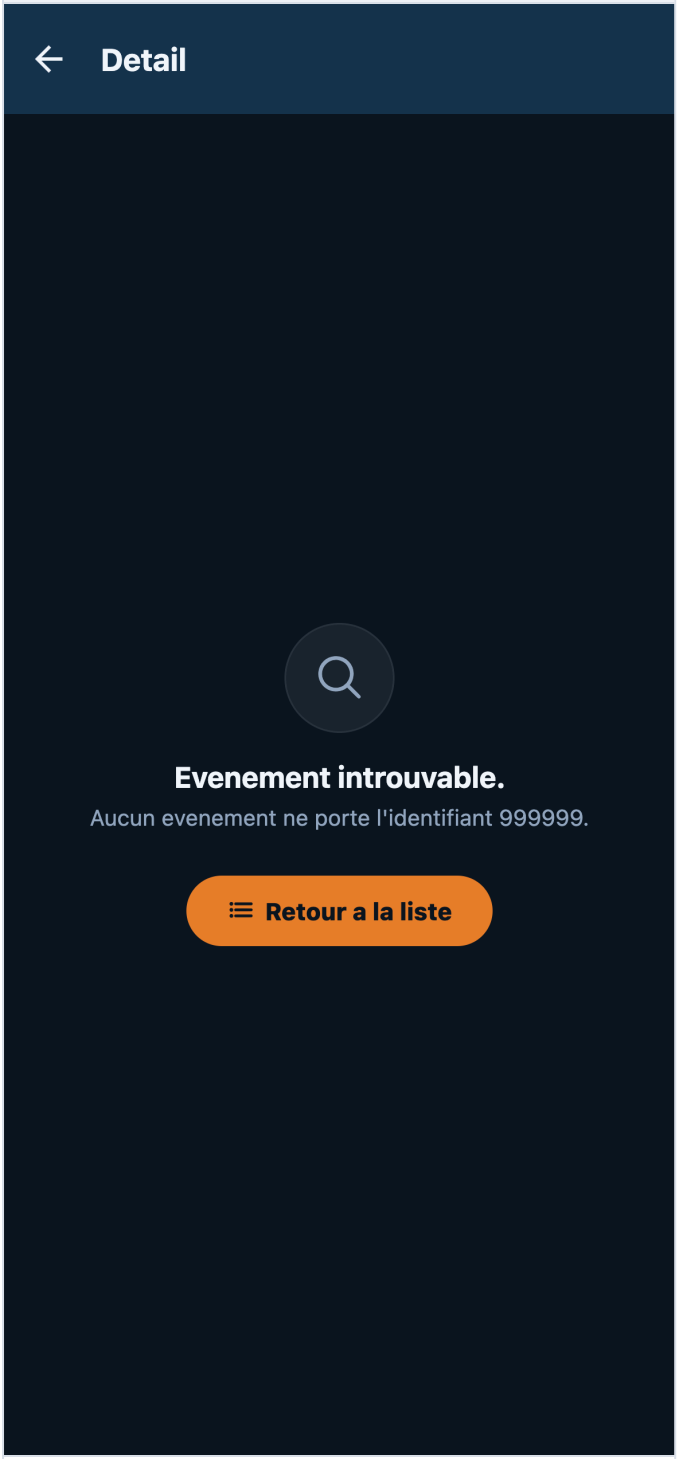
Lien direct



Ouvert directement par son URL, comme le fera un lien profond senevent://. Grace a initialRouteName, les onglets sont places dessous : la fleche retour existe et ramene a la liste.

FIGURE 05 `/evenement/999999`

Identifiant inconnu



Le .find() ne trouve rien : l'ecran le dit clairement et propose de revenir a la liste, au lieu d'un ecran blanc ou d'un spinner infini.

Vers le Lab 10

L'application a maintenant une vraie navigation, mais elle est en lecture seule. Au Lab 10, dernier lab mobile, l'onglet Profil accueillera la connexion (`seConnecter`, `sInscrire`, `seDeconnecter`), un écran de création utilisera `creerEvenement`, et l'écran de détail recevra le bouton d'inscription (`inscrire`, `desinscrire`) — toujours depuis le package partagé.